

Beginner Guide to Layouts & Data Classes in Jetpack Compose

In Jetpack Compose, layouts help us organize UI elements on the screen.

Think of layouts like arranging furniture in a room:

- A `Column` stacks items vertically
- A `Row` arranges items horizontally
- A `Box` places items on top of each other

Without layouts, UI components would appear messy and unorganized.

PART 1 — LAYOUTS IN JETPACK COMPOSE

What is a Layout?

A layout controls:

- Positioning
 - Alignment
 - Spacing
 - Arrangement of UI elements
-

Most Important Layouts

Layout	Purpose
<code>Column</code>	Vertical arrangement
<code>Row</code>	Horizontal arrangement
<code>Box</code>	Stack items

LazyColumn Scrollable vertical lists

LazyRow Scrollable horizontal lists

1. COLUMN LAYOUT

What is a Column?

A **Column** places items vertically from top to bottom.

Real World Example

Think of:

- Login form
- Settings menu
- Student list

Everything is stacked vertically.

Simple Column Example

```
package com.example.myapp
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material.Button
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.activity.compose.setContent
```

```
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            ColumnExample()
        }
    }
}

@Composable
fun ColumnExample() {

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {

        Text(text = "Welcome")

        Button(onClick = {}) {
            Text("Login")
        }

        Button(onClick = {}) {
            Text("Register")
        }
    }
}
```

Explanation

Column

Places items vertically.

`fillMaxSize()`

Makes Column occupy full screen.

`padding(16.dp)`

Adds spacing around content.

Expected Output

You will see:

- Welcome text
- Login button
- Register button

stacked vertically.

Column Arrangement

You can control spacing.

Example

`verticalArrangement = Arrangement.SpaceEvenly`

Other options:

- `Top`
 - `Bottom`
 - `Center`
 - `SpaceBetween`
 - `SpaceAround`
-

Column Alignment

Controls horizontal positioning.

```
horizontalAlignment = Alignment.CenterHorizontally
```

Full Example with Alignment

```
Column(  
    modifier = Modifier.fillMaxSize(),  
    verticalArrangement = Arrangement.Center,  
    horizontalAlignment = Alignment.CenterHorizontally  
) {  
  
    Text("Centered Text")  
  
}
```

Common Beginner Mistakes

1. Forgetting Modifier

Without modifiers, layouts may look compressed.

2. Confusing Arrangement and Alignment

Property	Controls
Arrangement	Main direction
Alignment	Opposite direction

For Column:

- Main direction = Vertical
 - Opposite = Horizontal
-

Beginner Exercises

1. Create 4 buttons vertically
 2. Center all items
 3. Add spacing between buttons
-

2. ROW LAYOUT

What is a Row?

A **Row** arranges items horizontally.

Real World Example

Think of:

- Toolbar buttons
 - Social media action buttons
 - Navigation menus
-

Row Example

```
package com.example.myapp
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material.Button
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.activity.compose.setContent
```

```
class MainActivity : ComponentActivity() {
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContent {
        RowExample()
    }
}

@Composable
fun RowExample() {

    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {

        Button(onClick = {}) {
            Text("Home")
        }

        Button(onClick = {}) {
            Text("Profile")
        }

        Button(onClick = {}) {
            Text("Settings")
        }
    }
}
```

Expected Output

Buttons appear horizontally.

Row Alignment

verticalAlignment = Alignment.CenterVertically

Important Concept

For **Row**:

- Main direction = Horizontal
 - Opposite direction = Vertical
-

Beginner Tasks

1. Create horizontal menu
 2. Add icons inside row
 3. Space buttons evenly
-

3. BOX LAYOUT

What is a Box?

A **Box** stacks components on top of each other.

Real World Examples

- Profile image with online badge
 - Text over image
 - Floating action button
-

Box Example

```
package com.example.myapplication
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.foundation.background
```

```
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.size
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.activity.compose.setContent

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            BoxExample()
        }
    }
}

@Composable
fun BoxExample() {

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.LightGray),
        contentAlignment = Alignment.Center
    ) {

        Text("Centered Inside Box")
    }
}
```

Expected Output

Centered text inside gray background.

Stacking Items Example

```
Box {  
  
    Image(...)  
  
    Text("New")  
}
```

Text appears on top of image.

Common Beginner Mistakes

1. Using Box Instead of Column

Remember:

- Box = stacking
 - Column = vertical
 - Row = horizontal
-

4. SPACER

What is Spacer?

Creates empty space between components.

Example

```
Spacer(modifier = Modifier.height(20.dp))
```

Full Example

```
Column {  
  
    Text("Top")  
  
    Spacer(modifier = Modifier.height(20.dp))  
  
    Text("Bottom")  
}
```

Expected Output

20dp spacing between texts.

5. NESTED LAYOUTS

What are Nested Layouts?

Layouts inside other layouts.

Real World Example

Instagram Post:

- Column
 - Row (profile section)
 - Image
 - Row (action buttons)
-

Nested Layout Example

```
Column {  
  
    Row {  
  
        Text("Profile")  
  
        Text("Username")  
    }  
  
    Text("Post Content")  
}
```

Why Nested Layouts Matter

Modern apps combine:

- Columns
- Rows
- Boxes

to create complex UI.

PART 2 — DATA CLASSES IN KOTLIN

What is a Data Class?

A data class stores data.

Think of it like a digital container.

Real World Analogy

A student form stores:

- name
- age
- grade

A product stores:

- name
 - price
 - image
-

Why Data Classes Are Important

Apps work with data:

- users
- products
- messages
- courses
- orders

Data classes organize this information cleanly.

Basic Data Class Syntax

```
data class Student(  
    val name: String,  
    val age: Int,  
    val grade: String  
)
```

Explanation

Keyword	Meaning
<code>data</code> <code>class</code>	Special class for storing data
<code>val</code>	Read-only value
<code>String</code>	Text
<code>Int</code>	Numbers

Creating Objects

```
val student1 = Student(  
    name = "John",  
    age = 20,  
    grade = "A"  
)
```

Accessing Data

```
println(student1.name)
```

Output:

John

Real World Applications of Data Classes

App Type	Data Class Example
WhatsApp	Message
Instagram	Post
Uber	Driver
School App	Student

Banking App Transaction

E-commerce Product

Example 1 — Product Data Class

```
data class Product(  
    val name: String,  
    val price: Double,  
    val image: Int  
)
```

Example Usage

```
val shoe = Product(  
    name = "Nike Air",  
    price = 1200.0,  
    image = R.drawable.shoe  
)
```

Example 2 — User Profile

```
data class User(  
    val username: String,  
    val email: String,  
    val profileImage: Int  
)
```

Data Classes with LazyColumn

This is where Compose becomes powerful.

Full Real World Example

```
package com.example.myapp
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.Card
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
```

```
data class Student(
    val name: String,
    val course: String
)
```

```
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            StudentScreen()
        }
    }
}
```

```
@Composable
fun StudentScreen() {

    val students = listOf(

        Student("John", "Android"),
        Student("Mary", "Cybersecurity"),
        Student("Kevin", "AI")

    )

    LazyColumn {

        items(students) { student ->
```

```
Card(  
    modifier = Modifier.padding(8.dp)  
) {  
  
    Text(  
        text = "${student.name} - ${student.course}",  
        modifier = Modifier.padding(16.dp)  
    )  
}  
}  
}
```

What Happens Here?

Step 1

Create Student data class.

Step 2

Create student objects.

Step 3

Display them using LazyColumn.

Expected Output

Cards displaying:

- John - Android
 - Mary - Cybersecurity
 - Kevin - AI
-

Why Data Classes Are Powerful

They:

- organize data
 - reduce code duplication
 - make apps scalable
 - simplify APIs and databases
-

Common Beginner Mistakes

1. Forgetting **data**

Wrong:

```
class Student(...)
```

Correct:

```
data class Student(...)
```

2. Using Wrong Data Types

Wrong:

```
val age: String
```

Correct:

```
val age: Int
```

3. Mixing UI and Data

Data classes should store data only.

Beginner Practice Tasks

Task 1

Create a **Movie** data class:

- title
 - year
 - rating
-

Task 2

Display movies using LazyColumn.

Task 3

Create a **Teacher** data class:

- name
 - subject
 - image
-

MINI PROJECT IDEAS

Project 1 — Student Management App

Use:

- Data classes
 - LazyColumn
 - Navigation
-

Project 2 — Food Delivery App

Use:

- Product data classes
 - Images
 - LazyRow
-

Project 3 — Social Media Feed

Use:

- Nested layouts
 - Box
 - Row
 - Column
 - Posts data class
-

BEGINNER INTERVIEW QUESTIONS

1. What is a Column?
 2. Difference between Row and Column?
 3. What is a Box used for?
 4. What is a data class?
 5. Why use data classes?
 6. What is nesting in Compose?
 7. What does Spacer do?
-

QUIZ QUESTIONS WITH ANSWERS

Q1

Which layout stacks items vertically?

Answer:

Column

Q2

Which layout stacks items on top of each other?

Answer:

Box

Q3

Why use data classes?

Answer:

To organize and store related data cleanly.

CODING CHALLENGES

Challenge 1

Create:

- Product data class
 - Product list
 - Product cards
-

Challenge 2

Create a dashboard using:

- Column
 - Row
 - Box
-

Challenge 3

Build:

- Student profile cards

- Images
 - Icons
 - LazyColumn
-

FINAL BEGINNER ADVICE

Master these concepts first:

1. Column
2. Row
3. Box
4. Modifiers
5. Data classes

These are the foundation of:

- modern Android apps
- clean UI design
- scalable applications
- professional Jetpack Compose development